Submitted to the *Journal of Computer Graphics Techniques* 2026-08-23

Kinematic Restitution Balancing (KRB)

A Per-Constraint Energy Audit for Velocity-Level Impulse Solvers

Joshua Sideris
Gear3Games
https://gearbox2d.com
josh.sideris@gmail.com

Abstract

Velocity-level sequential-impulse solvers gain energy from two analytic leaks: restitution and joint bias see the post-force velocity, and Baumgarte displacement does potential-energy work without a matching kinetic-energy tax. Kinematic Restitution Balancing (KRB) is a per-constraint `preSolve` audit of those leaks: Component A subtracts the force increment from the impact velocity, and Component B adjusts launch or bias speed for the expected correction Δh. A compile-time on/off ablation in Gearbox2D keeps a single elastic floor bounce at $E/E_0 \approx 1$ over $600\,\mathrm{s}$ (KRB off finishes at 6.53) and holds a high-rate enclosure in-box; a Newton's cradle remains a bounded offset, not an invariant. The method adds no solver passes.

1. Introduction

Velocity-level sequential-impulse (SI) solvers [Catto 2005] are the standard real-time rigid-body approach: constraints are solved after explicit force integration with a fixed iteration count. KRB is a drop-in *preSolve* energy audit for that pipeline. It does not add solver passes, replace PGS, or switch to position-based dynamics.

Bender, Erleben, and Trinkle survey interactive rigid-body practice [Bender et al. 2014]. Baumgarte stabilization [Baumgarte 1972] and split impulse / NGS position passes [Catto 2014, 2024] hide bounce-from-correction but do not tax PE work of the remaining Δh; TGS, soft constraints, and XPBD [Catto 2011; NVIDIA Corporation 2024; Macklin et al. 2016; Müller et al. 2007] damp or sidestep velocity-level bias at different cost. Variational integrators [Hairer et al. 2006] bound unconstrained drift; KRB is not one of those. Among the SI variants cited here, none apply an analytic PE/KE balance to expected Δh for both contacts and distance joints inside a single-pass impulse solver.

Two analytic sources inject artificial mechanical energy during constraint resolution.

Force-integration drift. With symplectic Euler integration, velocity is updated before the constraint solve:

$$v_{\text{solver}} = v_t + a_{\text{ext}} \cdot \Delta t. \tag{1}$$

Restitution and joint bias see v_{solver} instead of the true relative velocity v_{impact}. For a perfect bounce ($e = 1$), the launch velocity gains $a_{\text{ext}} \cdot \Delta t$ extra speed per frame. For a joint at rest, the solver sees gravity-induced velocity and fights it every frame at the velocity level.

Potential-energy shift from position correction. Baumgarte stabilization [Baumgarte 1972] feeds constraint error back as a velocity bias $v_{\text{bias}} = \beta C/\Delta t$, displacing bodies by Δh to resolve penetration or joint drift. That displacement increases potential energy (mgh) without reducing kinetic energy. The added PE converts to KE on later frames, causing runaway energy growth.

KRB applies that PE/KE balance to expected Δh at constraint setup for contacts and distance joints. The audit is per-constraint; coupled graphs can still offset (Section 6).

2. Method

KRB performs two independent corrections during constraint `preSolve`.

2.1. Component A: Force velocity compensation

With symplectic Euler, integration updates velocity before the constraint solve (Equation 1), so restitution and joint bias see v_{solver} rather than the pre-force relative velocity. Store the per-body force increment $v_{\text{force}} = a_{\text{ext}} \cdot \Delta t$ during integration and subtract it from the relative velocity seen at setup. That recovers the true impact normal velocity before restitution or bias is applied:

$$v_{\text{impact}} = v_{\text{relative}} - (v_{\text{force},B} - v_{\text{force},A}). \tag{2}$$

The correction is independent of the bias method.

2.2. Component B: Kinematic energy balancing

Consider a dynamic body against a static floor, contact normal $\mathbf{n}$ as in $v = (v_B - v_A) \cdot \mathbf{n}$, with center-of-mass motion along $\mathbf{n}$. If the position solver will raise the body by an expected distance Δh (Section 2.3; predicted Baumgarte displacement this step, not the work of the impulses the solver actually applied), free-flight rewind to the surface is $v_{\text{impact}}^2 = v_{\text{surf}}^2 - 2(\mathbf{a} \cdot \mathbf{n})\Delta h$, hence $v_{\text{surf}}^2 = v_{\text{impact}}^2 + 2(\mathbf{a} \cdot \mathbf{n})\Delta h$.

Equivalently, $\Delta E_p = -m(\mathbf{a}\cdot\mathbf{n})\Delta h$ taken from $\frac{1}{2}mv^2$ yields the same update: $\Delta(v^2) = -2\Delta E_p/m = 2(\mathbf{a}\cdot\mathbf{n})\Delta h$, so m cancels. The implementation applies this identity to the relative normal speed with $\mathbf{a}_{\text{rel}} = \mathbf{a}_B - \mathbf{a}_A$ (Listing 1, `forceVn`/Δt). When body A is static, $\mathbf{a} = \mathbf{a}_{\text{rel}}$. If both bodies share the same external acceleration, $\mathbf{a}_{\text{rel}} = \mathbf{0}$ and Component B is a no-op, consistent with an inverse-mass-weighted correction leaving pair-COM height unchanged. The identity does not include rotation, tangent motion, or impulse work; those residuals are the coupled-graph gap (Section 6). When available kinetic energy cannot pay the tax, clamp with $\max(0,\cdot)$ and apply restitution $v_{\text{final}} = e\, v_{\text{surf}}$:

$$v_{\text{surf}} = \sqrt{\max\big(0,\, v_{\text{impact}}^2 + 2(\mathbf{a}_{\text{rel}}\cdot\mathbf{n})\Delta h\big)}. \tag{3}$$

For ground collisions ($\mathbf{a}_{\text{rel}}\cdot\mathbf{n} < 0$), Component B taxes launch velocity to pay for increased PE; for ceiling collisions ($\mathbf{a}_{\text{rel}}\cdot\mathbf{n} > 0$), it credits velocity for work against external forces.

2.3. Effective displacement prediction

Δh in Equation (3) is a per-class *predictor* of this step's Baumgarte travel—not the full constraint error, and not a measurement of the impulses `solvePosition` actually applied. Contacts and distance joints share that identity; they do not share a predictor.

For bouncing contacts the velocity bias is restitution, so Component B predicts the later position pass. After slop s, the kinematic bounce $v_{\text{launch}} = -e\cdot v_{\text{impact}}$ partially resolves overlap before that pass. The remaining overlap is the effective depth

$$d_{\text{eff}} = \max\big(0,\, (d-s) - (v_{\text{launch}}\cdot\Delta t)\big). \tag{4}$$

Equation (4) uses that pre-tax launch. Substituting the post-tax $v_{\text{final}} = e\, v_{\text{surf}}$ from Equation (3) would make Δh implicit; we keep the explicit predictor. On a ground tax, $|v_{\text{final}}| < |v_{\text{launch}}|$, so d_{eff} is slightly low and the tax is slightly light—same sign as the original leak, and $O(\Gamma\,|a_{\text{rel}}|\Delta t/|v_{\text{impact}}|)$ when d_{eff} is interior. If $v_{\text{launch}}\Delta t$ already clears $(d-s)$, then $d_{\text{eff}} = 0$ and there is no loop. Many engines clamp per-iteration correction (e.g. Gearbox2D `MAX_POSITION_CORRECTION`). The contact predictor is then

$$\Delta h = \min(d_{\text{eff}}, \Delta h_{\max})\cdot\Gamma, \tag{5}$$

where $\Gamma = 1-(1-\beta)^n$ is the isolated geometric series (remaining error $\times(1-\beta)$ each iteration) over n position iterations with factor β. It is not a fit to coupled NGS. When $d_{\text{eff}} > \Delta h_{\max}$ every iteration, real travel is closer to $n\beta\Delta h_{\max}$ than $\Gamma\Delta h_{\max}$; the floor bounce does not hit that regime. Distance joints use the bias travel βC (Section 3.2), not Equation (5).

3. Constraint application

3.1. Contacts and speculative gaps

For physical overlap ($d > 0$), both Component A and Component B are required.

Speculative contacts are created before overlap ($d < 0$, a gap) to prevent tunneling. Because no position-correction work occurs yet, $\Delta h = 0$ and Component B is omitted; Component A remains critical so gravity-induced velocity is not treated as impact speed.

Apply restitution bias only when restitution is enabled and either the corrected normal velocity exceeds a small threshold, or a speculative gap is closing faster than the gap width allows. Resting contacts must not receive an $e = 1$ launch bias.

3.2. Distance joints and the zero-velocity paradox

Distance joints target zero relative velocity along the rod. Omitting Component B entirely leaks energy when Baumgarte stretch correction does work against gravity. Component B must be applied, but capped so a resting joint is not paralyzed.

Let $v_{\text{bias}} = \beta C/\Delta t$ be the ideal correction velocity for constraint error C. The audited correction is

$$v_{\text{bias\_actual}} = \sqrt{\max\bigl(0,\, v_{\text{bias}}^2 - 2(\mathbf{a}_{\text{rel}} \cdot \mathbf{n})(\beta C)\bigr)}. \tag{6}$$

Equation (6) is Equation (3) with joint predictor $\Delta h_{\text{joint}} = \beta C$: the travel implied by $v_{\text{bias}} = \beta C/\Delta t$, not Γ and not the position pass. Slop, the per-step $\Delta h_{\max}$ clamp, and extra joint position iterations when contacts overlap are uncaptured PE work (already inside the cradle residual). If kinetic energy cannot pay the PE tax, the joint retains a microscopic Baumgarte sag—bounded audit rather than infinite stiffness at rest. This paper's setup recipe covers contacts and *distance* joints; other joint types are outside the listed implementation.

4. Implementation

KRB runs at contact and distance-joint `preSolve`, before the velocity iteration loop. Prerequisites: symplectic Euler with per-body `forceVelocity` ($a_{\text{ext}}\Delta t$), Baumgarte position correction with factor β, position iteration count n, and penetration slop.

Integrate store. Per dynamic body at integrate (before the constraint solve), store v_{force} (`forceVelocity`): $v_{\text{force}} = a_{\text{ext}}\Delta t$ (two floats). Here a_{ext} is gravity plus external force before linear damping; the store is zero for sleeping or infinite-mass bodies.

4.1. Contact setup

```
// restitution threshold = 0.01; maxPosCorr = 0.2 (dH max)
forceVn = dot(bodyB.forceVelocity - bodyA.forceVelocity, n)
relativeVn = vn - forceVn
vBounce = -e * relativeVn
shouldBounce = enableRestitution &&
  (relativeVn < -RESTITUTION_THRESHOLD ||
   (depth < 0 && relativeVn < depth / dt))
if (shouldBounce) {
  if (depth > 0) {
    depthAfterVelocity = max(0, (depth - slop) - vBounce * dt)
    cumulativeFactor = 1 - pow(1 - beta, n)
    expectedDisplacement =
      min(depthAfterVelocity, maxPosCorr) * cumulativeFactor
  } else { expectedDisplacement = 0 }
  workTerm = 2 * (forceVn / dt) * expectedDisplacement
  vFinal = e * sqrt(max(0, relativeVn * relativeVn + workTerm))
  bias = (depth < 0) ? -max(vFinal, depth / dt) : -vFinal
} else if (depth < 0) { bias = -depth / dt } else { bias = 0 }
```

Listing 1. Contact `preSolve` bias (Component A + B).

4.2. Distance-joint setup

```
forceVn = dot(bodyB.forceVelocity - bodyA.forceVelocity, n)
v_bias_ideal = beta * C / dt
expectedDisplacement = v_bias_ideal * dt
workTerm = 2 * (forceVn / dt) * expectedDisplacement
v_bias_sq = v_bias_ideal * v_bias_ideal
v_bias_actual = sqrt(max(0, v_bias_sq - workTerm))
bias = (v_bias_ideal > 0 ? v_bias_actual : -v_bias_actual) - forceVn
```

Listing 2. Distance-joint `preSolve` bias (Component A + B).

During the velocity solve, $-\mathrm{forceVn}$ is folded into `bias` so `relative_vn` is not double-corrected (see the supplement distance-joint listing).

5. Evaluation

Four datasets (Table 1): A–B 600 s energy; C KRB-on 8 h ($t = 28800$ s); D native `world.step()` wall-time. Protocol: $dt = 1/60$, $e = 1$, zero friction and damping, sleep off; Datasets A and C use stock `World()` plus `constants.h` (`velocity_iterations`= 8, `position_iterations`= 3, $\beta = 0.2$, slop 0.016, restitution threshold 0.01). Dataset B Gearbox2D rows are a native offline recapture (`logEnergy --format b`, translational E/E_0); Box2D-WASM, p2.js, and Matter.js remain unmodified whole-engine WASM traces. The ablation switch is full KRB on/off. $E = E_k + E_p$ (Datasets A and C include rotational KE; Dataset B

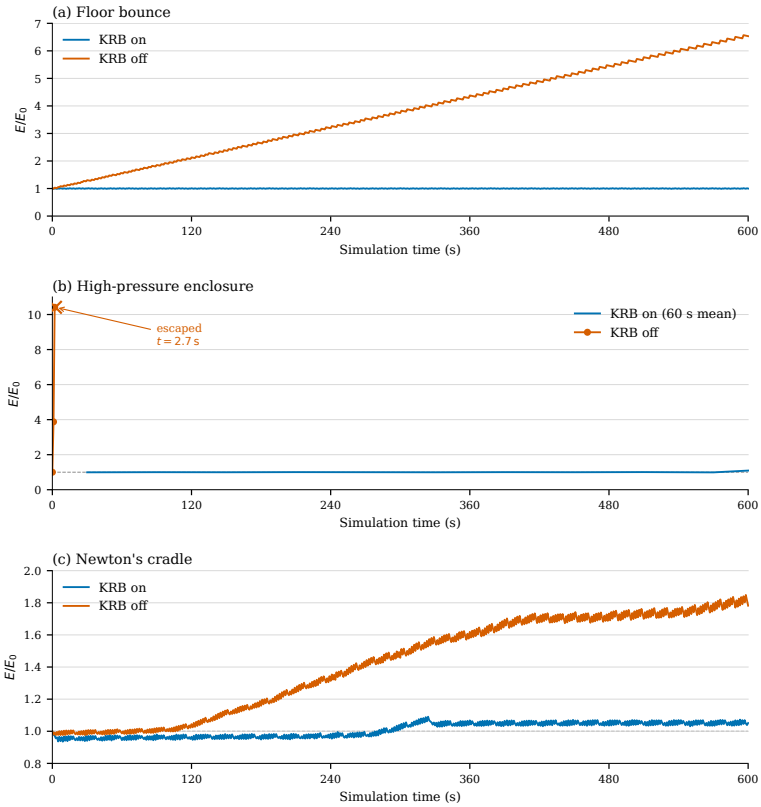

Figure 1. Mechanical energy ratio E/E_0 over $600\,\mathrm{s}$. (a) Elastic floor contact. (b) High-rate enclosure; KRB on is a $60\,\mathrm{s}$ windowed mean, KRB off raw $1\,\mathrm{Hz}$ samples to tunneling ($t = 2.7\,\mathrm{s}$). (c) Newton's cradle.

is translational only); the reported ratio is E/E_0.

Table 1. Evaluation protocol. Dataset C cells are KRB-on $8\,\mathrm{h}$ runs; enclosure E/E_0 uses $600\,\mathrm{s}$ windows of that trace.

Set	Scene	Horizon	Surface	Outcome
A	all	600 s	`log-energy`	Table 2
B	floor, cradle	600 s	offline Gearbox + WASM others	Table 3
C	enclosure	8 h	`log-energy`	in box; 600 s win. 1.006 → 0.976
C	cradle	8 h	`log-energy`	[1.031, 1.074]; mean 1.052
D	cradle + stack	timed steps	native `g++`	Table 4

Figure 2 continues the KRB-on traces to $8\,\mathrm{h}$.

Figure 3 and Table 3 place the default KRB-on Gearbox2D build next to unmodified Box2D-WASM, p2.js, and Matter.js. Iteration counts and damping differ, so this is whole-engine context, not an in-engine KRB ablation. Gearbox rows use the same translational meter as the website adapters, logged offline.

Table 2. Dataset A ablation: E/E_0 at $600\,\mathrm{s}$ (compile-time KRB on/off).

Scene	KRB	E/E_0	Win. mean	Outcome
Floor bounce	on	0.999	1.000	bounded
Floor bounce	off	6.53	climbing	runaway
High-pressure circle	on	—	1.00	stayed in box
High-pressure circle	off	—	—	escaped, step 161
Newton's cradle	on	1.05	1.05 after $t = 300$	bounded offset
Newton's cradle	off	1.79	climbing	secular gain

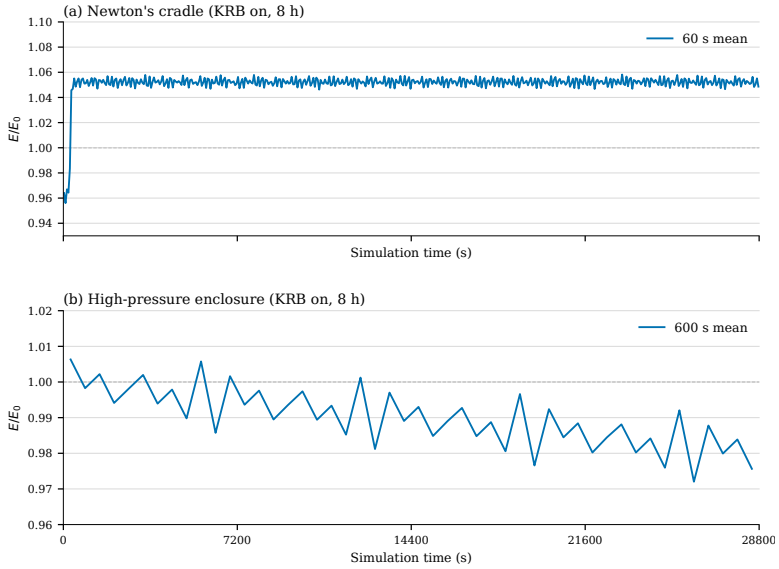

Figure 2. Dataset C KRB-on continuation to $8\,\mathrm{h}$ ($t = 28800\,\mathrm{s}$): windowed E/E_0 means (cradle $60\,\mathrm{s}$, enclosure $600\,\mathrm{s}$).

Table 3. Dataset B: E/E_0 at $600\,\mathrm{s}$. Gearbox2D native recapture; other engines unmodified.

Scene	Engine	E/E_0	Outcome
Floor bounce	Gearbox2D (KRB on)	0.999	bounded
Floor bounce	Box2D-WASM	6.61	runaway
Floor bounce	p2.js	0.376	slow loss
Floor bounce	Matter.js	≈ 0	dead by $t \approx 160\,\mathrm{s}$
Newton's cradle	Gearbox2D (KRB on)	1.051	bounded offset
Newton's cradle	Box2D-WASM	0.221	stepped loss
Newton's cradle	p2.js	0.320	slow loss
Newton's cradle	Matter.js	≈ 0	dead by $t \approx 40\,\mathrm{s}$

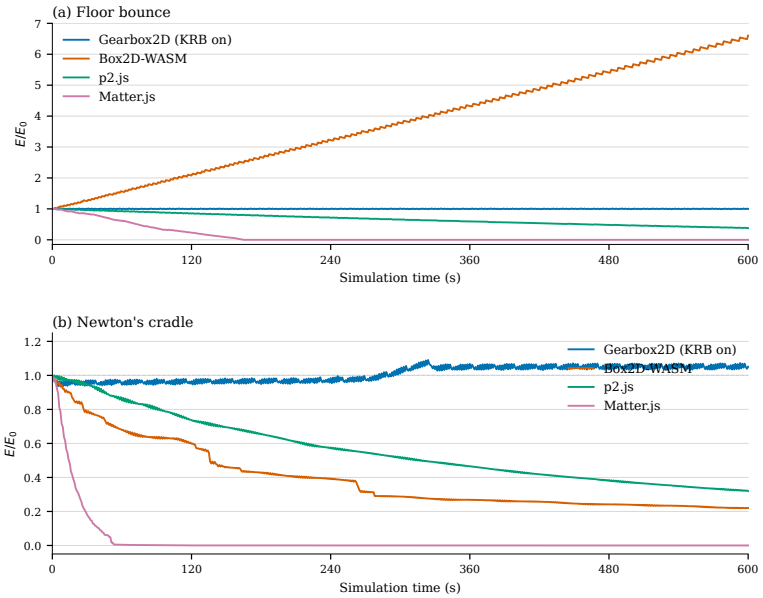

Figure 3. Mechanical energy ratio E/E_0 over $600\,\mathrm{s}$. Gearbox2D is a native KRB-on recapture (translational meter); Box2D-WASM, p2.js, and Matter.js are unmodified website exports. (a) Elastic floor contact. (b) Newton's cradle.

5.1. Cost (Dataset D)

Per dynamic body, KRB stores $\mathrm{forceVelocity} = a_{\mathrm{ext}} \cdot \Delta t$; each contact or distance-joint `preSolve` adds a forceVn dot, optional Γ, a `workTerm`, one `sqrt`, and a bias update. There are no extra PGS, velocity, or position iterations. Table 4 reports native `g++` on/off wall-time (`make log-timing`).

Table 4. Dataset D: native `g++` on/off wall-time per `world.step()` (mean and repeat range, ms/step). The stack is the contact-heavy scene and is not Dataset A. Traces: supplement `data/timing/`.

Scene	KRB on	KRB off
Newton's cradle	0.051 (0.050–0.052)	0.052 (0.050–0.053)
Large stack	0.448 (0.445–0.453)	0.426 (0.419–0.432)

Paper scenes sit in timer scatter; the denser stack is about $+5\%$ mean `world.step()` with non-overlapping repeats, and that is whole-step wall time, not isolated `preSolve`.

6. Limitations

KRB is a per-constraint audit, not a coupled one: resolving one constraint can force work on another, as when a horizontal collision lifts a pendulum rod. In the Newton's cradle that shows up as a step from ~ 0.96 to ~ 1.05 near four minutes (Figure 1c); the $8\,\mathrm{h}$ band is in Table 1 and Figure 2a. The Dataset C enclosure stays in-box; $600\,\mathrm{s}$ windowed E/E_0 drifts

$1.006 \rightarrow 0.976$ (Figure 2b), a slow loss of about $0.030\,E_0$ rather than the KRB-off SI gain of Figure 1b.

Energy results use the protocol in Section 5: single-pass SI/PGS, symplectic Euler, contacts and distance joints, $e = 1$, zero friction and damping, sleep off, and the stock World defaults and `constants.h` knobs. Friction, $e < 1$, other dt, and other iteration counts were not measured.

A coupled-constraint audit, tunneling, and long ill-conditioned chains remain future work, as do other integrators, hinge joints, and a switch to XPBD or TGS. Variational integrators [Hairer et al. 2006] are a different claim.

7. Conclusion

KRB is a `preSolve` audit for sequential-impulse solvers [Catto 2005]: Component A subtracts symplectic-Euler force drift from the impact velocity, and Component B taxes the expected correction Δh. Contact scenes in the Gearbox2D ablation stay bounded; the coupled-constraint gap is in Section 6.

References

BAUMGARTE, J. Stabilization of constraints and integrals of motion in dynamical systems. *Computer Methods in Applied Mechanics and Engineering*, 1(1):1–16, 1972. URL: https://doi.org/10.1016/0045-7825(72)90018-7. 1, 2

BENDER, J., ERLEBEN, K., AND TRINKLE, J. Interactive simulation of rigid body dynamics in computer graphics. *Computer Graphics Forum*, 33(1):246–270, 2014. URL: https://doi.org/10.1111/cgf.12272. 1

CATTO, E. Iterative dynamics with temporal coherence. Presented at the Game Developers Conference, San Francisco, CA, 2005. URL: https://box2d.org/files/ErinCatto_IterativeDynamics_GDC2005.pdf. 1, 9

CATTO, E. Soft constraints. Presented at the Game Developers Conference, San Francisco, CA, 2011. URL: https://box2d.org/files/ErinCatto_SoftConstraints_GDC2011.pdf. 1

CATTO, E. Understanding constraints. Presented at the Game Developers Conference, San Francisco, CA, 2014. URL: https://box2d.org/files/ErinCatto_UnderstandingConstraints_GDC2014.pdf. 1

CATTO, E. Solver2D. Box2D blog post, February 2024. URL: https://box2d.org/posts/2024/02/solver2d/. 1

HAIRER, E., LUBICH, C., AND WANNER, G. *Geometric Numerical Integration: Structure-Preserving Algorithms for Ordinary Differential Equations*, volume 31 of *Springer Series in Computational Mathematics*. Springer-Verlag, Berlin, Germany, 2 edition, 2006. URL: https://doi.org/10.1007/3-540-30666-8. 1, 9

MACKLIN, M., MÜLLER, M., AND CHENTANEZ, N. XPBD: Position-based simulation of compliant constrained dynamics. In *Proc. 9th ACM SIGGRAPH Conference on Motion in*

Games (MIG), pages 49–54, Burlingame, CA, 2016. URL: https://doi.org/10.1145/2994258.2994272. 1

MÜLLER, M., HEIDELBERGER, B., HENNIX, M., AND RATCLIFF, J. Position based dynamics. *Journal of Visual Communication and Image Representation*, 18(2):109–118, 2007. URL: https://doi.org/10.1016/j.jvcir.2007.01.005. 1

NVIDIA CORPORATION. Rigid body dynamics. PhysX SDK 5.4 Documentation, 2024. URL: https://nvidia-omniverse.github.io/PhysX/physx/5.4.1/docs/RigidBodyDynamics.html. 1

Index of Supplemental Materials

The archive `krb-supplement.zip` (ISC) is distributed with this article. Paths are relative to the zip root: `README.md`; `listings/contact-presolve.cpp`; `listings/distance-joint-presolve.cpp`; `listings/integrate-force-velocity.cpp`; `data/dataset-a/`; `data/dataset-b/`; `data/dataset-c/`; `data/timing/`; `plots/`; `figures/`. Recapture uses Gearbox2D tag `krb-jcgt-1` at https://github.com/JSideris/Gearbox2D.

Author Contact and License

Joshua Sideris, Gear3Games — josh.sideris@gmail.com, gearbox2d.com. Supplemental code and data in `krb-supplement.zip` are ISC-licensed.

Joshua Sideris, Kinematic Restitution Balancing: A Per-Constraint Energy Audit for Velocity-Level Impulse Solvers, *Journal of Computer Graphics Techniques (JCGT)*, vol. ?, no. ?, 1–1, ????
????

Received:	2026-08-23		
Recommended:	????-??-??	Corresponding Editor:	??? ???
Published:	????-??-??	Editor-in-Chief:	Alexander Wilkie